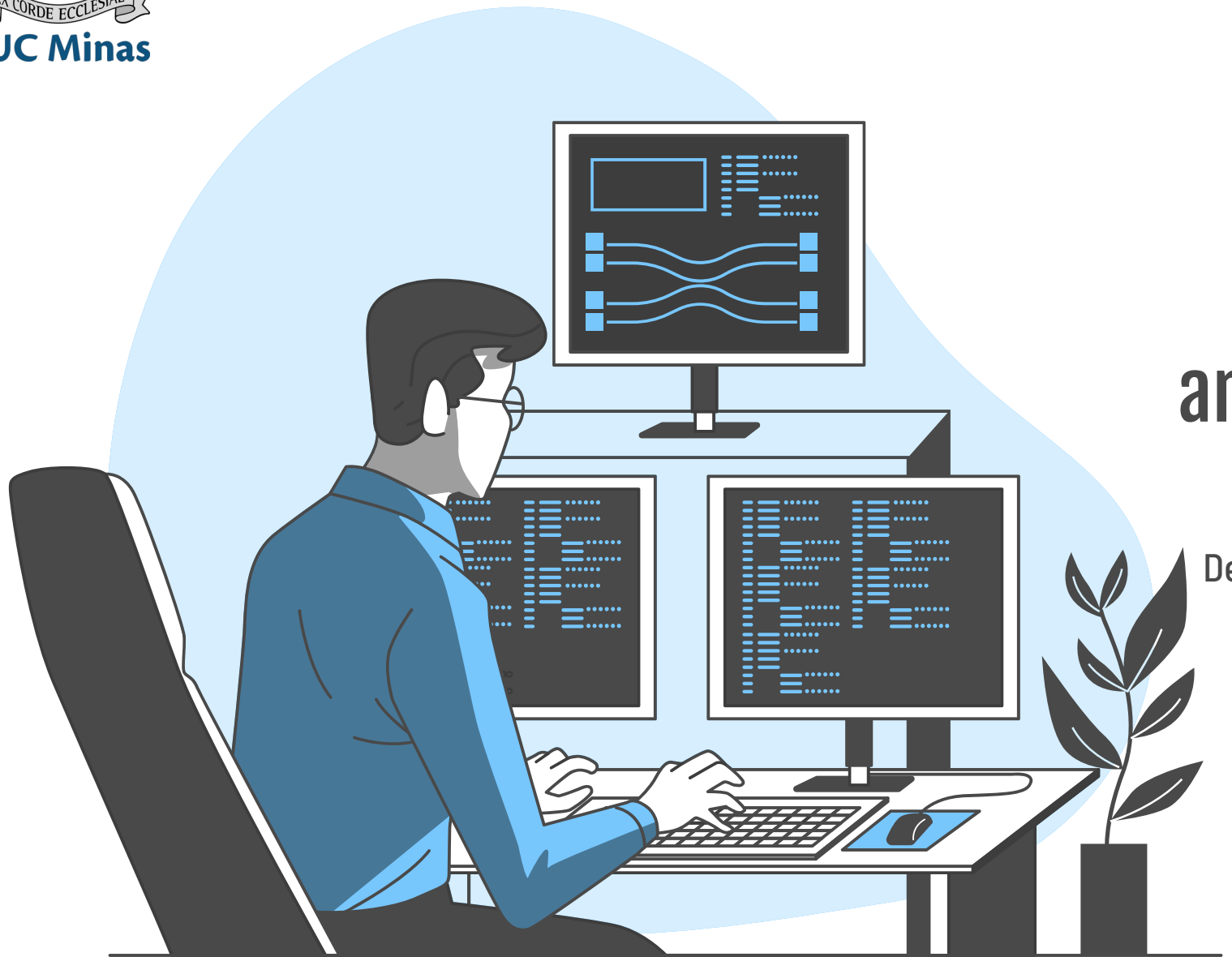
PUC Minas



Aula 07 · Front-end

Variáveis de ambiente: .env, .env.local e VITE_

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender por que **não** escrever URLs e chaves fixas no código

2

Conhecer a ordem de leitura de `.env`, `.env.local`, `.env.development`, `.env.production`

3

Usar o prefixo `VITE_` e `import.meta.env`

4

Saber o que **nunca** pode ir para o front-end

5

Versionar um `.env.example` e manter `.env.local` fora do Git

Apresentação da aula

Antes de começar

- Aula 06: services e config

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O problema

02 Os arquivos que o Vite lê

03 Regras do Vite

04 Variáveis prontas do Vite

05 E no Vercel?

06 Aprofundando: ambientes na prática

01

O problema

O problema

No seu computador o back-end está em `http://localhost:8080`. No Vercel, ele está em `https://minha-api.onrender.com`. Se a URL estiver escrita no código, você precisa editar e lembrar de voltar a cada deploy. A solução é ler o valor de uma **variável de ambiente**, que muda conforme o lugar onde o projeto roda.

01

O problema

Aula 07 · Front-end

02

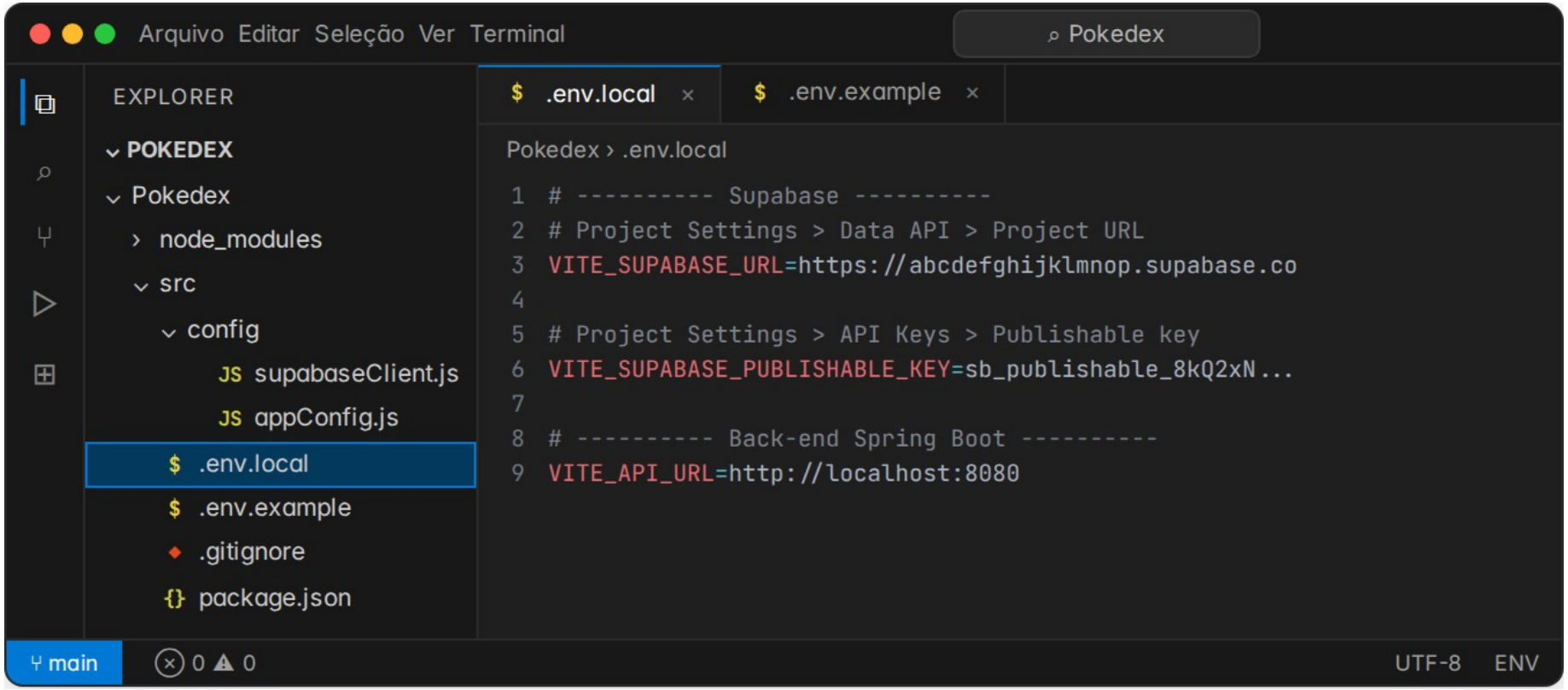
Os arquivos que o Vite lê

Os arquivos que o Vite lê

Arquivo	Carregado em	Vai para o Git?
`.env`	Sempre	Pode, se não tiver segredos
`.env.local`	Sempre, **sobrescreve** o `.env`	**Não** (já está no `.gitignore` do Vite: `*.local`)
`.env.development`	Só no `npm run dev`	Pode
`.env.production`	Só no `npm run build`	Pode
`.env.development.local` / `.env.production.local`	Idem, com prioridade máxima	**Não**
`.env.example`	**Nunca** (é só documentação)	**Sim**: mostra quais variáveis o projeto precisa

Os arquivos que o Vite lê

Prioridade, da maior para a menor: variáveis já definidas no sistema (ex.: painel do Vercel) → `.env.[modo].local` → `.env.[modo]` → `.env.local` → `.env`.



`.env.local` com os valores reais (fora do Git). O `.env.example` tem as mesmas chaves com valores fictícios.

03

Regras do Vite

Regras do Vite

1. Só variáveis que **começam com** `VITE_` chegam ao código. `API_URL=...` é ignorada (proteção contra vazar segredos sem querer).
2. No código, leia com `import.meta.env.VITE_NOME` (no CRA era `process.env.REACT_APP_NOME`).
3. Todo valor é **texto**: `VITE_DEBUG=false` chega como a string `"false"`.
4. Alterou o `.env.local`? **Reinicie** o `npm run dev`.
5. No build, o Vite **substitui** `import.meta.env.VITE_X` pelo valor literal dentro do JavaScript. Mudou a variável no Vercel? Faça um **novo deploy**.

03

Regras do Vite

Aula 07 · Front-end

Regras do Vite

src/config/apiConfig.js (projeto Exemplo_Integracao)

```
/**
 * - No seu computador: http://localhost:8080 (sem configurar nada).
 * - No Vercel: cadastre VITE_API_URL com a URL do Render.
 */
const URL_PADRAO = "http://localhost:8080";

const API_CONFIG = {
  // remove a barra final, se houver ("...com/" vira "...com")
  BASE_URL: (import.meta.env.VITE_API_URL || URL_PADRAO).replace(/\/+$/, ""),
};

export default API_CONFIG;
```

Regras do Vite

.env.example (vai para o Git)

```
# URL do back-end Spring Boot.
# No Vercel, cadastre em Settings >
# Environment Variables com a URL
# do Render (sem barra no final):
VITE_API_URL=https://seu-back.onrender.com
```

.gitignore

```
node_modules
dist
dist-ssr
*.local
.env
```

04

Variáveis prontas do Vite

Variáveis prontas do Vite

Variável	Valor
`import.meta.env.MODE`	`"development"` ou `"production"`
`import.meta.env.DEV`	`true` no `npm run dev`
`import.meta.env.PROD`	`true` no build
`import.meta.env.BASE_URL`	Valor de `base` no `vite.config.js`

Variáveis prontas do Vite

O que é seguro no front-end

Tudo que está no front-end é público. O build é um arquivo JavaScript que qualquer pessoa baixa e lê no DevTools, inclusive o valor das variáveis `VITE_*`. Por isso:

- **Pode** ir para o front: URL da API, URL do Supabase, a *publishable key* (antiga *anon key*) do Supabase, chave pública do reCAPTCHA, *public key* do Stripe e do EmailJS.
- **Nunca** pode: a *secret key / service_role* do Supabase, senha de banco, *connection string* do MongoDB, chave secreta do Stripe, `JWT_SECRET`, chaves de APIs pagas (OpenAI, Gemini) que gerem custo.

Segredos ficam **no back-end** (variáveis do Render), e o front chama o back-end.

Variáveis prontas do Vite

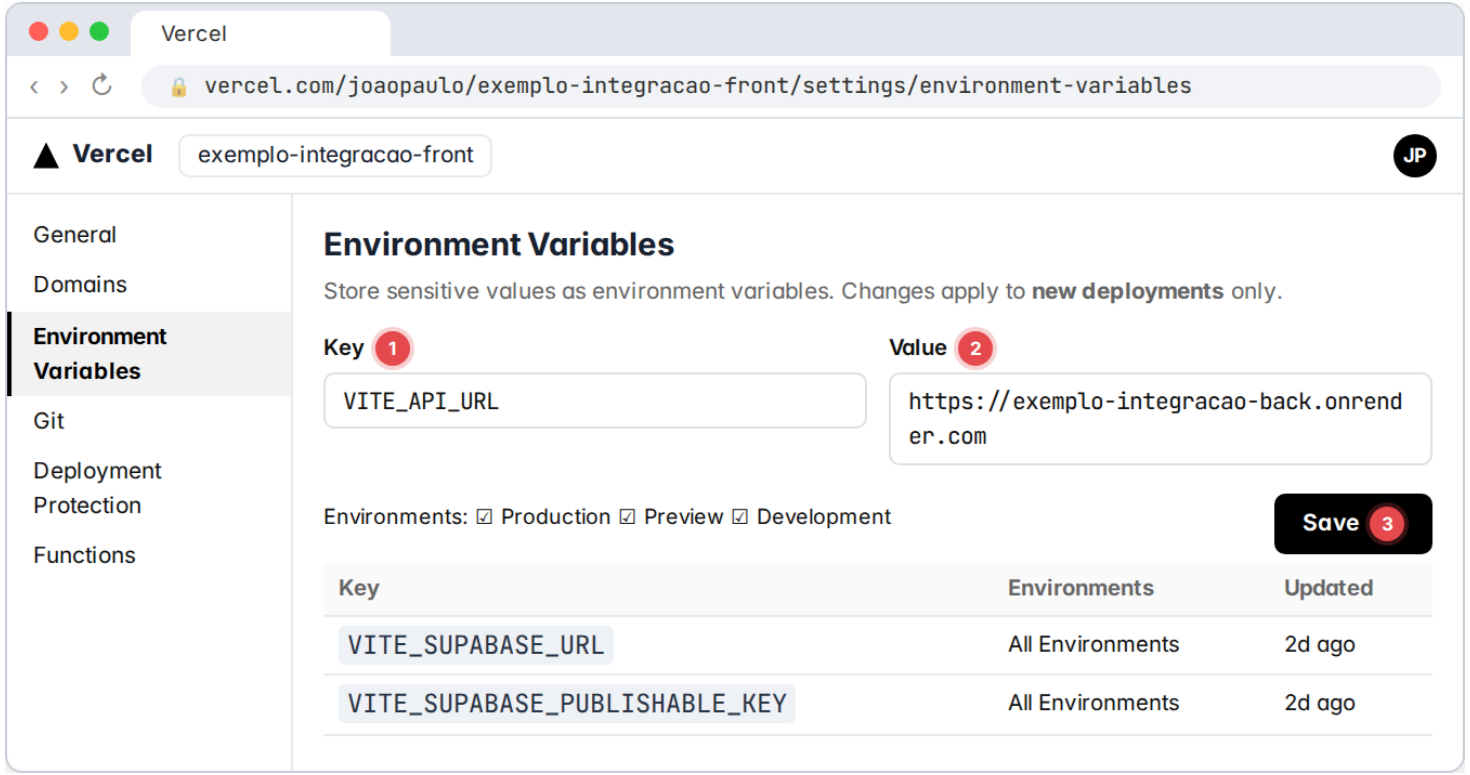
Erro comum

`import.meta.env.VITE_API_URL` é `undefined`: o nome não começa com `VITE_`; o arquivo está fora da raiz do projeto (deve ficar ao lado do `package.json`); ou o servidor não foi reiniciado. **Funciona local, mas não no Vercel**: a variável não foi cadastrada no painel, ou foi cadastrada depois do último deploy.

05

E no Vercel?

E no Vercel?



Em **Settings → Environment Variables** cadastra-se o equivalente ao `.env.local` (detalhes na pasta Deploy).

06

Aprofundando: ambientes na prática

Aprofundando: ambientes na prática

Modo	Ativado por	Arquivos lidos (em ordem de prioridade)
development	`npm run dev`	`.env.development.local`, `.env.development`, `.env.local`, `.env`
production	`npm run build`	`.env.production.local`, `.env.production`, `.env.local`, `.env`
staging (exemplo)	`vite build --mode staging`	`.env.staging.local`, `.env.staging`, `.env.local`, `.env`

Aprofundando: ambientes na prática

Um valor por ambiente

```
# .env.development
VITE_API_URL=http://localhost:8080

# .env.production
VITE_API_URL=https://meu-back.onrender.com

# .env.staging
VITE_API_URL=https://meu-back-staging.onrender.com
```

Validando as variáveis na inicialização

src/config/appConfig.js

```
const obrigatorias = ["VITE_SUPABASE_URL", "VITE_SUPABASE_PUBLISHABLE_KEY"];
const faltando = obrigatorias.filter((k) => !import.meta.env[k]);

if (faltando.length) {
  console.error(`Configure no .env.local: ${faltando.join(", ")}`);
}

export const config = {
  supabaseUrl: import.meta.env.VITE_SUPABASE_URL,
  supabaseKey: import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY,
  debug: import.meta.env.VITE_DEBUG === "true",   // texto → booleano
};
```

Variáveis do front x do back

Front (Vite, Vercel)

- Prefixo `VITE_` obrigatório
- Gravadas no JS no build: **públicas**
- Mudou? Novo build/Redeploy

vs

Back (Spring, Render)

- Qualquer nome (`MONGODB_URI`)
- Lidas em tempo de execução: **privadas**
- Mudou? Reiniciar o serviço

Mapa de variáveis de um projeto full stack

VITE_API_URL

Front → onde está a API. Local: vazio. Vercel: URL do Render.

VITE_SUPABASE_*

Front → URL e chave publicável, se o navegador fala com o Supabase.

MONGODB_URI / DB_*

Back → conexão com o banco. Só no Render.

CORS_ORIGINS

Back → quem pode chamar a API. Inclui a URL do Vercel.

Aprofundando: ambientes na prática

1. Crie o `.env.example` com todas as chaves e valores fictícios e envie para o Git.
2. Copie para `.env.local` e preencha os valores reais.
3. Confira que `.env.local` aparece como ignorado no `git status`.
4. Reinicie o `npm run dev`.
5. No Vercel, cadastre as mesmas chaves em *Settings* → *Environment Variables* e faça Redeploy.

Aprofundando: ambientes na prática

Erro comum

Colocou uma chave secreta em `VITE_...` e publicou? Ela está no JavaScript do site e qualquer um pode copiar. **Revogue a chave** no painel do serviço (Supabase, Stripe, Gemini) e gere outra; apagar do código não basta.

07

Quiz de revisão

Pergunta

Qual prefixo uma variável precisa ter para chegar ao código no Vite?

A REACT_APP_

B VITE_

C PUBLIC_

D ENV_

Resposta

Qual prefixo uma variável precisa ter para chegar ao código no Vite?

A REACT_APP_

✓ VITE_

C PUBLIC_

D ENV_

Por quê: REACT_APP_ era o do Create React App.

Pergunta

Como ler uma variável no código Vite?

A process.env.X

B import.meta.env.VITE_X

C window.env.X

D require('env')

Resposta

Como ler uma variável no código Vite?

A process.env.X

✓ import.meta.env.VITE_X

C window.env.X

D require('env')

Por quê: process.env era o do CRA.

Pergunta

Qual arquivo guarda os valores reais e não vai para o Git?

A .env.example

B .env.local

C package.json

D vite.config.js

Resposta

Qual arquivo guarda os valores reais e não vai para o Git?

A .env.example

✓ .env.local

C package.json

D vite.config.js

Por quê: O .env.example documenta as chaves e vai para o Git.

Quiz · questão 4 de 5

Pergunta

Mudou `VITE_API_URL` no Vercel. O que fazer?

A Nada

B Redeploy: o valor é gravado no JS durante o build

C Reiniciar o navegador

D Apagar o cache do npm

Quiz · questão 4 de 5

Resposta

Mudou `VITE_API_URL` no Vercel. O que fazer?

A Nada

✓ Redeploy: o valor é gravado no JS durante o build

C Reiniciar o navegador

D Apagar o cache do npm

Por quê: Variáveis do Vite são resolvidas no build.

Pergunta

Qual destas pode ir no front-end?

A service_role do Supabase

B Senha do MongoDB

C Publishable key do Supabase

D JWT_SECRET

Resposta

Qual destas pode ir no front-end?

A service_role do Supabase

B Senha do MongoDB

✓ Publishable key do Supabase

D JWT_SECRET

Por quê: Tudo no front é público; segredos ficam no back.

Fechamento

Resumo da aula

- ✓ URLs e chaves saem do código e vão para variáveis de ambiente.
- ✓ `.env.local` tem os valores reais e **não** vai para o Git; `.env.example` documenta as chaves e **vai**.
- ✓ Só variáveis `VITE_*` chegam ao código, lidas com `import.meta.env`.
- ✓ Os valores são gravados no JS durante o build: mudou no Vercel, faça novo deploy.
- ✓ Tudo no front é público: segredos ficam no back-end.

Exercícios

1. Mova a URL da API do seu projeto para `VITE_API_URL` com valor padrão no `apiConfig.js`.

2. Crie `.env.development` e `.env.production` com URLs diferentes e confira com `npm run dev` e `npm run build && npm run preview`.

3. Abra o build no DevTools (aba Sources) e encontre o valor da sua variável dentro do JS.

4. Classifique como “pode” ou “não pode ir para o front”: 10 chaves fornecidas pelo professor.

Para ir além

- vite.dev/guide/env-and-mode
- Projetos `Pokedex`, `SecureLoginPUC_React` e `FullStack/Exemplo_Integracao/Front` (veja os `.env.example`)

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 07?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web